

Memory

Advanced Programming
Brent van Bladel



Variables

A **variable** is an abstraction for a memory space.

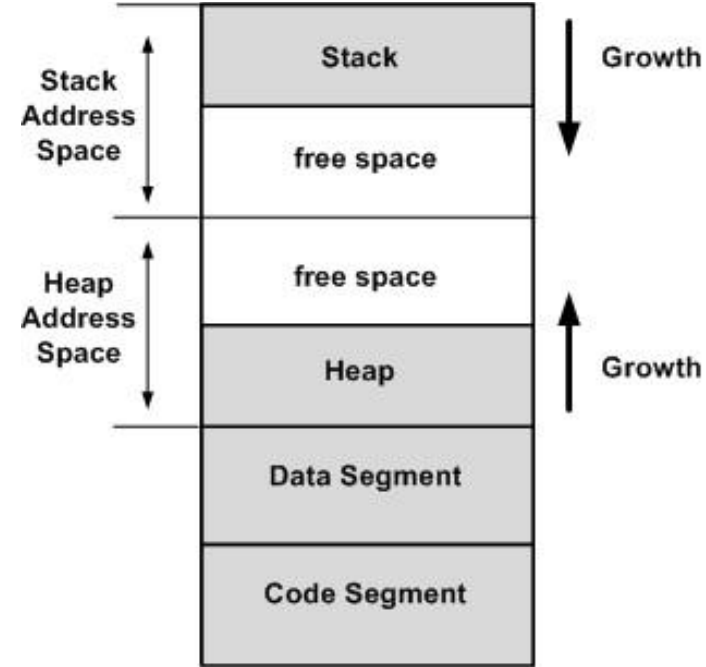
Attributes

- **Name:** how to refer to the memory in the program text
- **Address:** where in memory the variable is stored
- **Type:** how to interpret the binary data in memory
- **Storage duration:** rules determining when the memory is available
- **Lifetime:** when the memory can be referenced consistently

Stack vs Heap

Stack is used for local variables, function arguments, and control information, such as return addresses.

The **heap** is a flexible area for storing objects with dynamic lifetimes.



Storage Duration

The **storage duration** is a variable's property that defines the rules according to which an object's memory is allocated and freed.

Four options

- **Automatic:** from start of scope until end of scope
- **Static:** from start of program until end of program
- **Thread:** from start of thread until end of thread
- **Dynamic:** from allocation request until deallocation request

Lifetime

The **lifetime** is the time period during the execution of the program in which the variable can be referenced consistently.

Typically, the lifetime of a variable is from when its memory is allocated until that memory is released.

Variables

```
1. void f(){  
2.   int x = 42;  
3. }
```

- **Name:** x
- **Address:** 0x7ffc4c219cc
- **Type:** int
- **Storage duration:** automatic
- **Lifetime:** until out of scope

STACK

...	0	0	42	0	0	0	0	0	...
-----	---	---	----	---	---	---	---	---	-----

0x7ffc4c219cc

Variables

```
1. void f(){  
2.     int x = 42;  
3. }
```

Integer (x) stored on the stack

Storage duration: automatic

Lifetime: from initialization until out of scope

```
1. void f(){  
2.     int* x = new int{42};  
3. }
```

Integer (42) stored on the heap

Storage duration: dynamic

Lifetime: from 'new' until 'delete'

Integer pointer (x) stored on the stack

Storage duration: automatic

Lifetime: from initialization until out of scope

Variables

```
1. void f(){  
2.     int* x = new int{42};  
3. }
```

- **Name:** x
- **Address:** 0x31684e456ef
- **Type:** int*
- **Storage duration:** automatic
- **Lifetime:** until out of scope

STACK

0x7ffc4c219cc

0

...

0x31684e456ef

HEAP

...

0

0

42

0

0

0x7ffc4c219cc

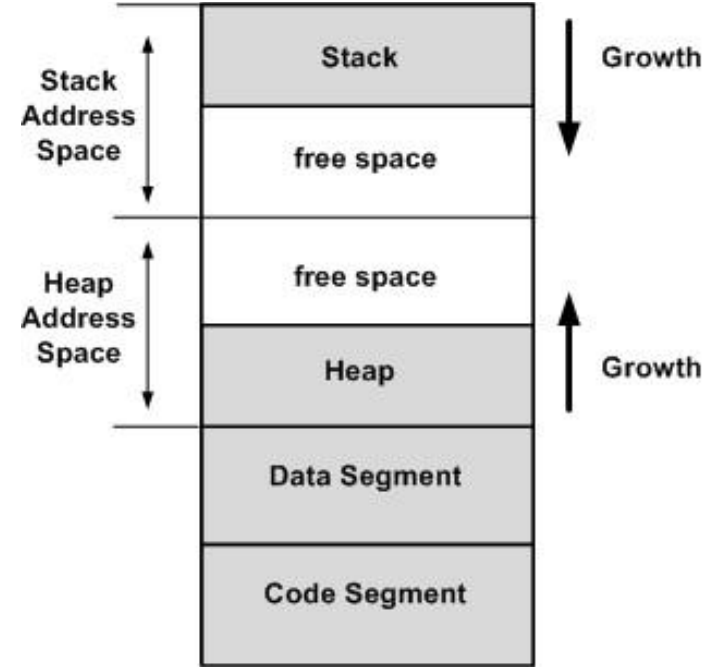
- **Name:**
- **Address:** 0x7ffc4c219cc
- **Type:** int
- **Storage duration:** dynamic
- **Lifetime:** until 'delete' call

Stack vs Heap

Stack is used for local variables, function arguments, and control information, such as return addresses.

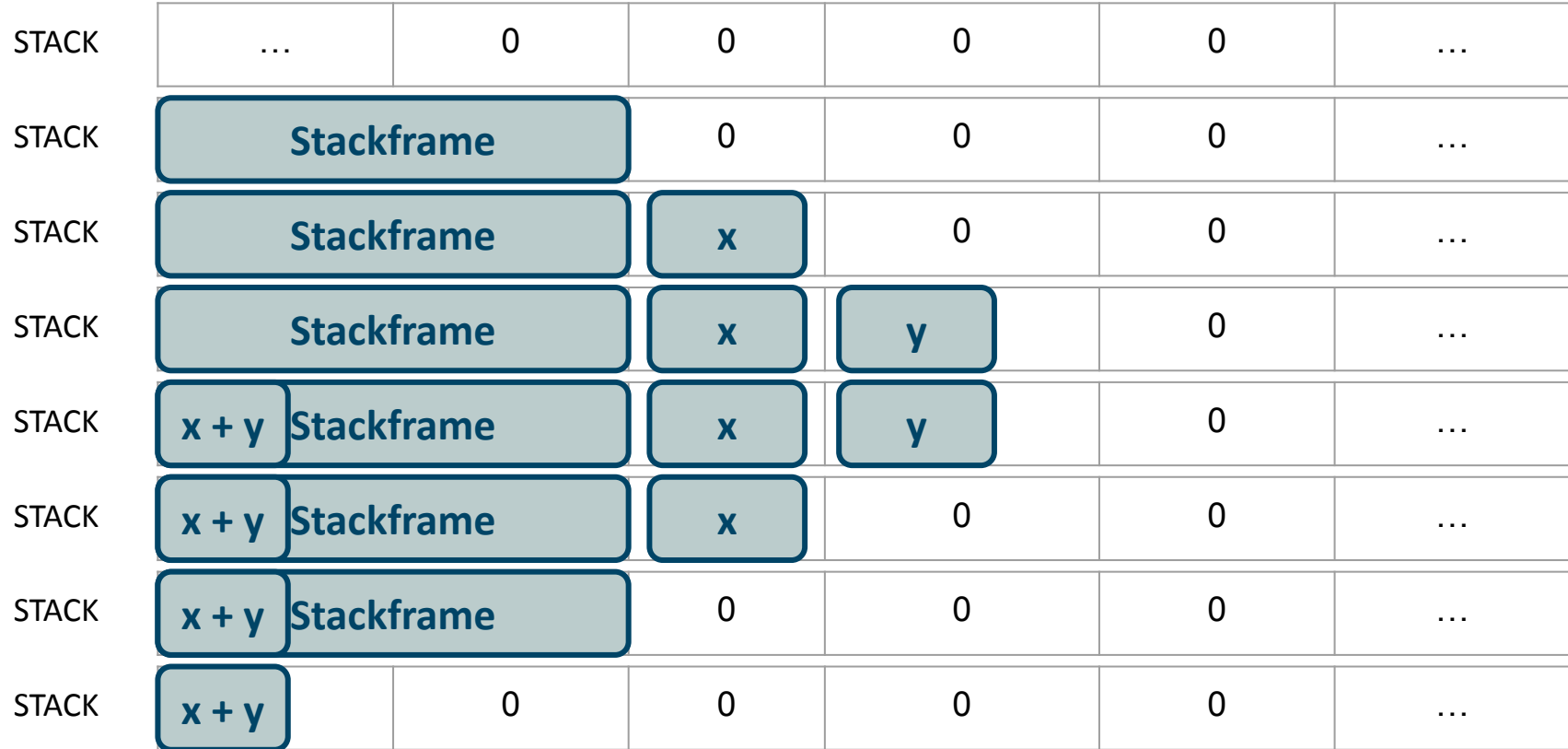
The **heap** is a flexible area for storing objects with dynamic lifetimes.

Note that allocating on the heap is more expensive compared to the stack!



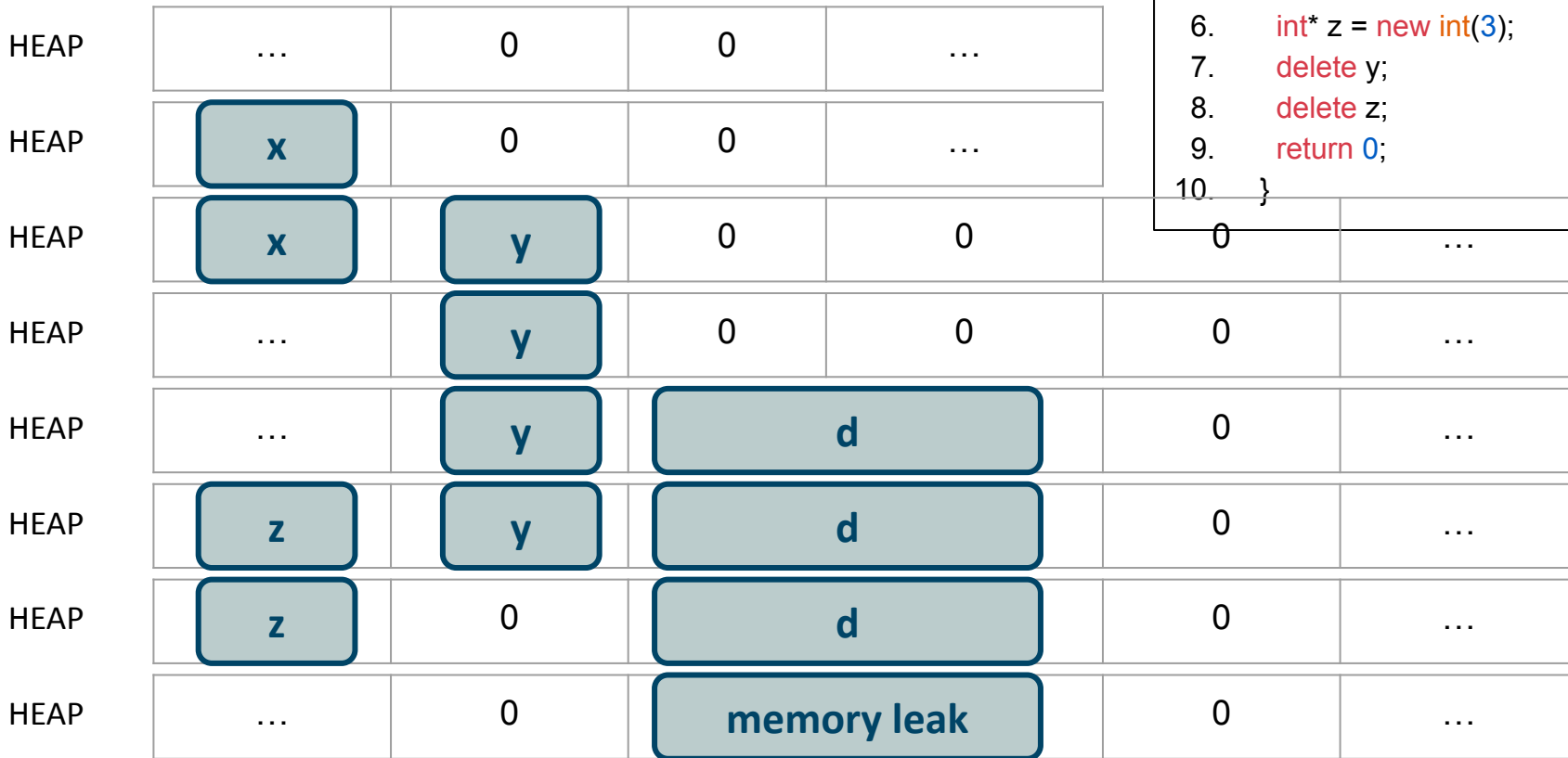
Stack Allocation Example

```
1.  int f() {  
2.    int x = 1; int y = 2;  
3.    return x + y;  
4.  }
```



Heap Allocation Example

```
1. int main(){  
2.     int* x = new int(1);  
3.     int* y = new int(2);  
4.     delete x;  
5.     double* d = new double(3.3);  
6.     int* z = new int(3);  
7.     delete y;  
8.     delete z;  
9.     return 0;  
10. }
```



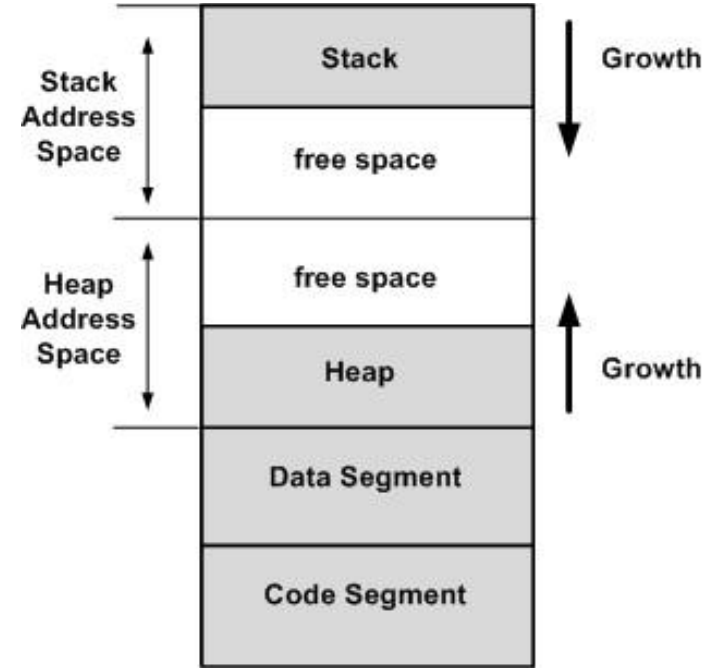
Stack vs Heap

Stack allocation is fast:

- Next available address is known
- Direct access

Heap allocation is expensive:

- Next available address is unknown
- Indirect access



Memory in C++

Quiz

How many bits does this code allocate?

```
1. bool b = true;
```

Answer: **8 bits (1 byte)**

Memory is aligned and addressed **per byte**.

How many bytes does this code allocate?

```
1. auto v = std::vector<int>{1,2,3};
```

Answer: **24 byte** on the stack & **12 byte** on the heap

Stack	Heap
8 byte address to start of vector	4 byte int (1)
8 byte address to end of vector	4 byte int (2)
8 byte address to end of capacity	4 byte int (3)

What is the output of this code?

```
1.  auto a = std::vector<int>{1,2,3,4,5};  
2.  
3.  cout << "Size of a: " << a.size() << endl;  
4.  cout << "Capacity of a: " << a.capacity() << endl;
```

Answer:

Size of a: 5

Capacity of a: 5

What is the output of this code?

```
1.  auto b = std::vector<int>{};
2.  b.push_back(1);
3.  b.push_back(2);
4.  b.push_back(3);
5.  b.push_back(4);
6.  b.push_back(5);
7.
8.  cout << "Size of b: " << b.size() << endl;
9.  cout << "Capacity of b: " << b.capacity() << endl;
```

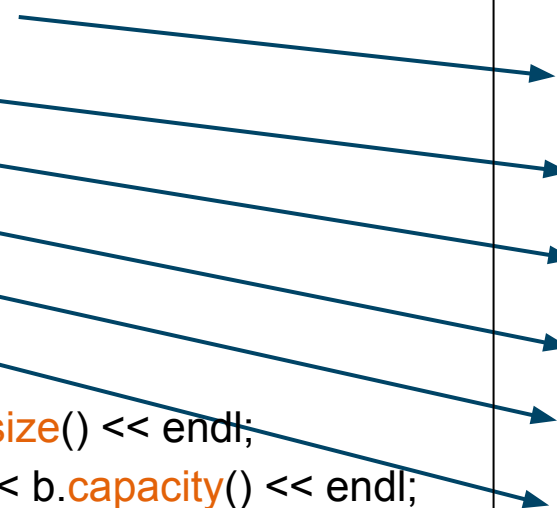
Answer:

Size of b: 5

Capacity of b: 8

Vector Capacity

```
1. auto b = std::vector<int>{};  
2. b.push_back(1);  
3. b.push_back(2);  
4. b.push_back(3);  
5. b.push_back(4);  
6. b.push_back(5);  
7.  
8. cout << "Size of b: " << b.size() << endl;  
9. cout << "Capacity of b: " << b.capacity() << endl;
```



Size	Capacity
0	0
1	1
2	2
3	4
4	4
5	8



Capacity increase == copy of the entire vector

What is the output of this code?

```
1.  auto c = std::vector<int>{};
2.  c.reserve(5);
3.  c.push_back(1);
4.  c.push_back(2);
5.  c.push_back(3);
6.  c.push_back(4);
7.  c.push_back(5);
8.
9.  cout << "Size of c: " << c.size() << endl;
10. cout << "Capacity of c: " << c.capacity() << endl;
```

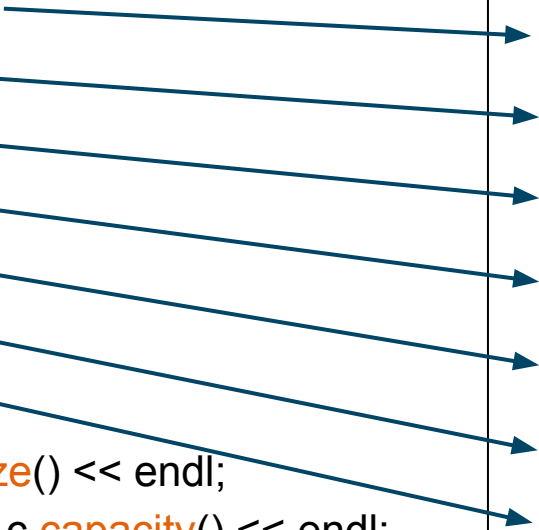
Answer:

Size of c: 5

Capacity of c: 5

Vector Capacity

```
1. auto c = std::vector<int>{};
2. c.reserve(5);
3. c.push_back(1);
4. c.push_back(2);
5. c.push_back(3);
6. c.push_back(4);
7. c.push_back(5);
8.
9. cout << "Size of c: " << c.size() << endl;
10. cout << "Capacity of c: " << c.capacity() << endl;
```



Size	Capacity
0	0
0	5
1	5
2	5
3	5
4	5
5	5

What is the output of this code?

```
1. char str[] = "Hello World";  
2. str[0] = "W";  
3. cout << str << endl;
```

Answer:

Wello World

What is the output of this code?

```
1. char* str = "Hello World";  
2. str[0] = "W";  
3. cout << str << endl;
```

Answer:

Runtime error!

Char Array vs Char Pointer

```
1. char str[] = "Hello World";  
2. str[0] = "W";  
3. cout << str << endl;
```

Allocates 12 chars on the stack:

{'H','e','l','l','o',' ','W','o','r','l','d', 0}

Stack Memory Usage: 12 byte

Note: char* is more efficient when passing a string to a function!

```
1. char* str = "Hello World";  
2. str[0] = "W";  
3. cout << str << endl;
```

Allocate 1 pointer on the stack.

This pointer points to a char array.

Stack Memory Usage: 8 byte

The char array is stored in static memory.

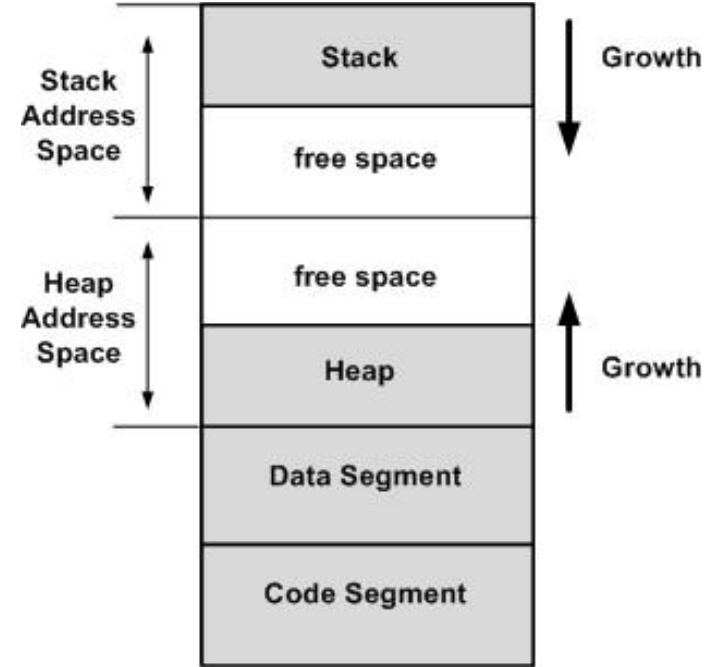
Static Memory Usage: 12 byte

Static Memory Area

The static area of memory is of fixed size.

It consists of 2 parts:

- **Read-only code:** the machine language program and constants, including string constants.
- **Static data:** global variables, which can be changed by the program, but always occupy a fixed number of bytes.



In which memory are the chars of this string stored?

```
1. std::string s = "Hello world, solar-system, and universe!";
```

Answer:

Heap

std::string implementation example

```
1. class string {  
2.     public:  
3.         // all public member functions  
4.     private:  
5.         size_type m_size;  
6.         size_type m_capacity;  
7.         char* m_data;  
8. };
```

STACK

...	m_size	m_capacity	0x7ffc4c219cc	0	...
-----	--------	------------	---------------	---	-----

0x31684e456ef

HEAP

...	0	0	'H'	'e'	'l'	'l'	'o'	' '	...
-----	---	---	-----	-----	-----	-----	-----	-----	-----

0x7ffc4c219cc

In which memory are the chars of this string stored?

```
1. std::string s = "Hi";
```

Answer:

Implementation dependent, but for most modern implementations: stack!

SSO (Short String Optimization)

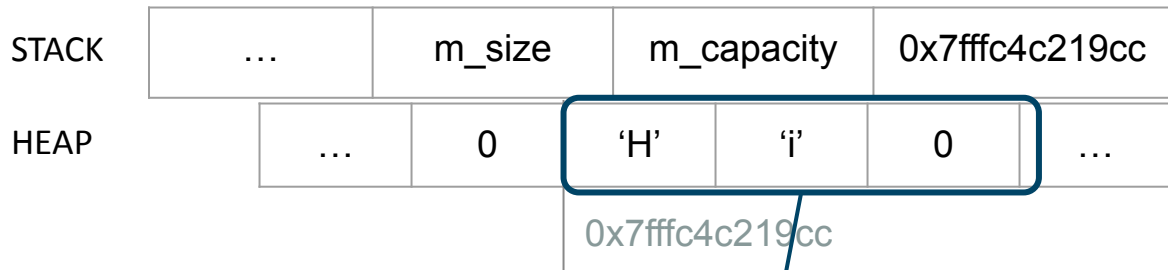
std::string implementation with SSO example

```
1.  class string {  
2.      public:  
3.          // all public member  
        functions  
4.      private:  
5.          size_type m_size;  
6.          union {  
7.              struct {  
8.                  size_type m_capacity;  
9.                  char* m_data;  
10.             } m_large;  
11.             char[] m_small;  
12.         }  
13. };
```

If the char-array is small enough to fit in the memory spots of the capacity member and the char* member, we use that memory on the stack to store the char array instead. In this case, there is no need for heap allocation.

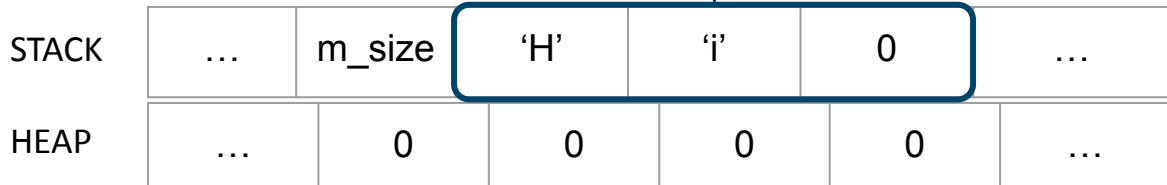
Otherwise, implementation will behave like the string without SSO.

Example: string "Hi"



Without SSO

m_capacity + char* == 12 byte
char [] == 3 byte



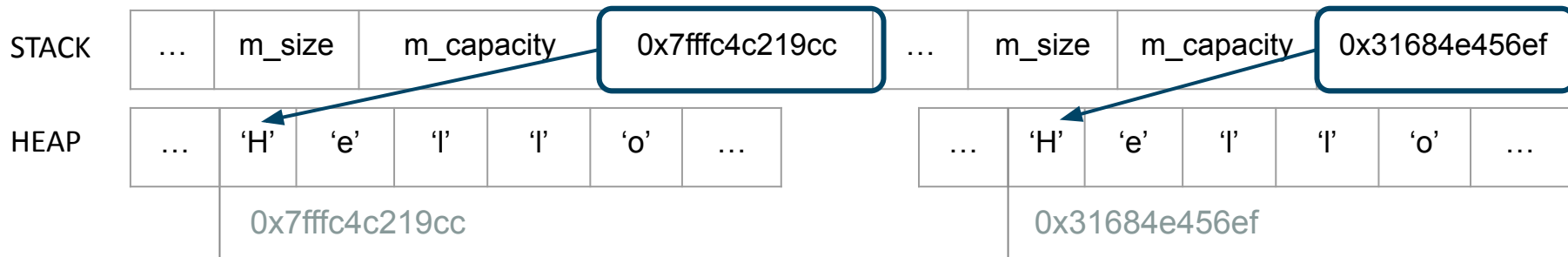
With SSO

Does a string copy also copy the heap memory?

1. `std::string str = "The quick brown fox jumps over the lazy dog.";`
2. `std::cout << str.substr(5, 19) << std::endl;`

Answer: Yes, a *deep copy* is made.

Deep copy



Shallow copy



Does a string copy also copy the heap memory?

1. `std::string str = "The quick brown fox jumps over the lazy dog.";`
2. `std::cout << str.substr(5, 19) << std::endl;`

Answer: Yes, a *deep copy* is made.

This is a very expensive operation.

Solution: `std::string_view`

std::string_view

```
1. class string_view {  
2.     size_type m_size;  
3.     const char* m_data;  
4. };
```

A **string_view** is a view on a string. It cannot be used to modify the string data.

However, construction and copy is cheap as no additional dynamic memory is allocated!

std::string_view

1. `std::string str = "The quick brown fox jumps over the lazy dog.";`
2. `std::cout << str.substr(5, 19) << std::endl;`



Using a string_view prevents additional dynamic memory usage!

1. `std::string str = "The quick brown fox jumps over the lazy dog.";`
2. `std::string_view sv = str;`
3. `std::cout << sv.substr(5, 19) << std::endl;`

std::string_view literal

```
1. void print(std::string_view msg) {  
2.     std::cout << msg << std::endl;  
3. }  
4.  
5. int main() {  
6.     print("Hello world"sv);  
7. }
```

If you need a string_view literal: put `sv` after your string literal.

What is the output of this code?

```
1. #include <iostream>
2. #include <sstream>
3. using namespace std;
4.
5. string_view concat(string_view s1, string_view s2){
6.     stringstream ss;
7.     ss << s1 << " " << s2;
8.     return ss.str();
9. }
10.
11. int main() {
12.     cout << concat("Hello"sv, "World"sv) << endl;
13. }
```

Answer: Undefined behaviour

Stringstream allocates a copy of the characters on the heap.

We return a string_view to those characters.

End of scope: stringstream destructor removes characters from the heap.

Dangling pointer!

Dangling Pointer

A **dangling pointer** is a pointer that outlives the lifetime of the object it points to.

Accessing a dangling pointer will always result in **undefined behaviour**.

What is the output of this code without SSO?

```
1.  string s = "Hi";  
2.  string_view sv{s};  
3.  s = "Hello";  
4.  cout << sv << endl;
```

Answer: Undefined behaviour

Reassignment requires new allocation on heap (likely in a different address), while the string_view pointer is not updated.

What is the output of this code with SSO?

```
1.  string s = "Hi";  
2.  string_view sv{s};  
3.  s = "Hello";  
4.  cout << sv << endl;
```

Answer: “He”

Both strings fit on the stack.

The string_view pointer remains valid, yet its size does not get updated.

What is the output of this code without SSO?

```
1.  string s = "Hello";  
2.  string_view sv{s};  
3.  s = "Hi";  
4.  cout << sv << endl;
```

Answer: “Hilo”

If the new string fits in the capacity, no memory gets allocated or freed.

What is the output of this code with SSO?

```
1.  string s = "Hello";  
2.  string_view sv{s};  
3.  s = "Hi";  
4.  cout << sv << endl;
```

Answer: “Hilo”

Which parameter uses the least memory?

```
1. void foo(int a, int& b, int* c);
```

Answer: a, the pass by value.

An integer allocates 32 bit, a pointer/reference allocates 64 bit!
For larger objects (> 64 bit), a pass by pointer or reference will be more efficient.

Pass by reference

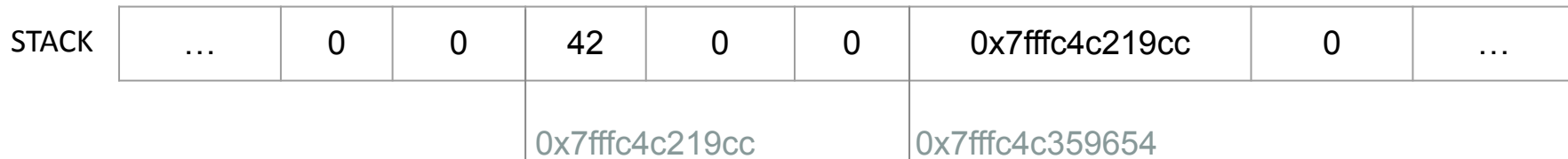
Note that the lifetime of b and x differ!

```
1. void foo(int& b);  
2.  
3. int main() {  
4.     int x = 42;  
5.     foo(x);  
6. }
```

Name: b
Address: 0x7ffc4c359654
Type: reference
Storage duration: automatic
Lifetime: until out of scope

Name: x
Address: 0x7ffc4c219cc
Type: int
Storage duration: automatic
Lifetime: until out of scope

A **reference** is an alias, an alternate name for an object.
On the stack it is passed as the address of an object (*pointer*).



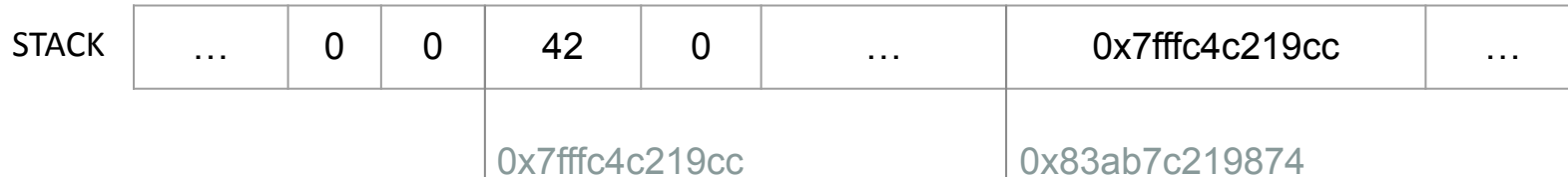
Pass by pointer (1)

```
1. void foo(int* c);  
2.  
3. int main() {  
4.     int x = 42;  
5.     foo(&x);  
6. }
```

Name: c
Address: 0x83ab7c219874
Type: int*
Storage duration: automatic
Lifetime: until out of scope

Name: x
Address: 0x7ffc4c219cc
Type: int
Storage duration: automatic
Lifetime: until out of scope

A **pointer** is a variable containing the address of an object.



Pass by pointer (2)

```
1. void foo(int* c);  
2.  
3. int main() {  
4.     int* x = new int{42};  
5.     foo(x);  
6. }
```

Name: c

Address: 0x83ab7c219874

Type: int*

Storage duration: automatic

Lifetime: until out of scope

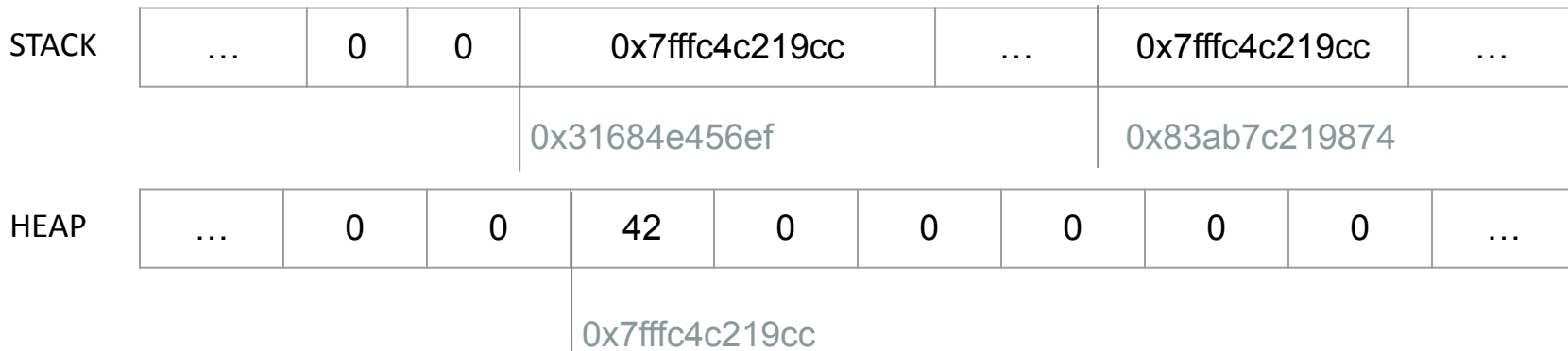
Name: x

Address: 0x31684e456ef

Type: int*

Storage duration: automatic

Lifetime: until out of scope



nullptr

```
1. void print(int* p){  
2.     if (p)  
3.         cout << *p << endl;  
4.     else  
5.         cout << "null" << endl;  
6. }  
7.  
8. int main() {  
9.     print(nullptr);  
10. }
```

Explicitly point to nothing.

Can be checked by conversion to boolean.

Note: only checks for nullptr, not if the pointer is dangling!

std::optional

```
1. void print(std::optional<int> o){
2.     if (o)
3.         cout << *o << endl;
4.     else
5.         cout << "empty" << endl;
6. }
7.
8. int main() {
9.     print(42);
10.    print({});
11.    print(std::nullopt);
12. }
```

Extends the type domain by an additional “null” value, without dynamic memory allocation.